SIMATS ONLINE PROGRAMMING LAB

PYTHON PROGRAMMING — ASSIGNMENT 3 TECHNICAL REPORT

Student Name	Pughzhvanan D
Register Number	1912260281
Course & Code	CSA0809 — Python Programming (CO3 Coverage)
Repository Format	CSA08-1912260281-Assignment3
Repository URL	https://github.com/ppughzhvanan-byte/CSA08-1912260281-Assignment3.git
Total Marks & Bloom's	50 Marks (Q1 to Q7) | K3 - Apply, K4 - Analyse, K5 - Evaluate

TABLE OF CONTENTS

1. **Q1:** Lists and Slicing: Daily Rainfall Records (6 Marks | K3 Apply)
2. **Q2:** Tuples, Mutability and Copying: City Bus-Stop Map (7 Marks | K4 Analyse)
3. **Q3:** Dictionaries: Farmer Helpline Complaint Analysis (7 Marks | K4 Analyse)
4. **Q4:** Sets: Elective Registration Clash Detection (7 Marks | K4 Analyse)
5. **Q5:** List Comprehension: Cleaning Sensor Readings (6 Marks | K3 Apply)
6. **Q6:** Nested Data: A Railway Timetable (7 Marks | K4 Analyse)
7. **Q7:** Design a Contact-Merging Program from First Principles (10 Marks | K5 Evaluate)

Q1 — Lists and Slicing: Daily Rainfall Records (6 Marks | K3 - Apply)

Problem Context: Village Rain-Gauge Station recording daily rainfall (mm) for 365 days.

1. Implementation & Python Code

```
import random

def analyse_rainfall(readings):
    if not readings:
        return {}, [], []

    # Weekly totals using slicing
    weekly_totals = [sum(readings[i:i+7]) for i in range(0, len(readings), 7)]

    # Max and Min with positions
    max_val = max(readings)
    min_val = min(readings)
    max_pos = [i for i, x in enumerate(readings) if x == max_val]
    min_pos = [i for i, x in enumerate(readings) if x == min_val]

    # 7-day moving average (Running sum approach)
    moving_avg = []
    if len(readings) >= 7:
        window_sum = sum(readings[:7])
        moving_avg.append(window_sum / 7.0)
        for i in range(7, len(readings)):
            window_sum += readings[i] - readings[i-7]
            moving_avg.append(window_sum / 7.0)

    # Dry days
    dry_days = [i for i, x in enumerate(readings) if x == 0]
    return weekly_totals, max_pos, min_pos, moving_avg, dry_days
```

2. Slicing Mechanics & Theoretical Analysis

- `readings[7:14]`: Extracts 2nd week readings (indices 7 to 13).
- `readings[-7:]`: Extracts the last 7 days of the year.
- `readings[:7]`: Extracts the first 7 days (index 0 to 6).
- **Addition Counts:** Naive slicing performs 6 additions per window $\times (N-6)$ windows $\approx 6N$. Running sum performs 7 additions initially plus 2 operations per step $\approx 2N$.

3. Alternative Concept & Complexity Analysis

Approach	Time Complexity	Space Complexity	Evaluation
List Slicing (Naive)	O(7n) = O(n)	O(n)	Recomputes sum of 7 elements at every step. Higher constant factor.
Running Sum List	O(n)	O(n)	Optimal execution speed; keeps all history in contiguous array.
collections.deque(maxlen=7)	O(n)	O(1) extra space	Best for streaming data where only current 7-day window is needed.

Q2 — Tuples, Mutability and Copying (7 Marks | K4 - Analyse)

Problem Context: City Bus-Stop Map storing GPS coordinates as `(lat, long)` tuples and zone grid passenger counts.

1. Complete Code Solution (TODO 1 Fixed)

```
import copy, sys

rows, cols = 3, 4
# BAD: Every row references the SAME list object in memory
bad = [[0] * cols] * rows
bad[0][1] = 7

# TODO 1: Create rows x cols grid of zeros using list comprehension
good = [[0] * cols for _ in range(rows)]
good[0][1] = 7

print("bad :", bad)
print("good:", good)
print(bad[0] is bad[1], good[0] is good[1])

# Tuples as Dictionary Keys
stops = {(13.0827, 80.2707): "Chennai Central", (13.0418, 80.2341): "T. Nagar"}
print(stops[(13.0418, 80.2341)])
```

2. Memory Diagrams & Hashability Explanation

```
BAD GRID: bad = [[0]*4]*rows bad ----> [ Row Pointer 0, Row Pointer 1, Row Pointer 2 ] | | | +--------
```

- **Hashability:** Tuples are immutable and implement `__hash__()`, remaining constant throughout their lifetime. Lists are mutable; modifying a list changes its logical state, which would corrupt dictionary bucket indexing. Thus, lists raise `TypeError: unhashable type: 'list'`.
- **Memory Benchmarking:** `sys.getsizeof(tuple_1M)` uses ~8MB whereas `sys.getsizeof(list_1M)` uses ~9MB due to dynamic allocation padding in lists.

Q3 — Dictionaries: Farmer Helpline Complaint Analysis (7 Marks | K4)

Problem Context: Kisan Call Centre text complaint processing and word ranking.

1. Python Implementation (TODO 1 & TODO 2 Complete)

```
calls = [
    "seeds not received seeds delayed",
    "fertiliser price high",
    "seeds price high subsidy not received"
]
STOP = {"not", "high"}

def frequencies(texts, stop):
    counts = {}
    for text in texts:
        for word in text.split():
            if word not in stop:
                counts[word] = counts.get(word, 0) + 1
    return counts

def top(counts, k):
    # Sort by count descending (-x[1]), then alphabetically ascending (x[0])
    sorted_items = sorted(counts.items(), key=lambda x: (-x[1], x[0]))
    return sorted_items[:k]

print(top(frequencies(calls, STOP), 3))
# Expected Output: [('seeds', 3), ('price', 2), ('received', 2)]
```

2. Analysis & Decision

- **Tie Breakdown:** Both `'price'` and `'received'` occur 2 times. The tuple key `(-count, word)` ranks count first. When counts tie (-2 == -2), string comparison `'price' < 'received'` puts `'price'` first.
- **Lookup Complexity:** Hash dictionaries access keys in average **O(1)** time via hash calculation. Scanning a list with `list.count()` for d distinct words over n total words takes **O(n · d)** time.
- **Deployment Choice:** Deploy `dict` / `collections.Counter` (O(n) total time) for processing 100,000 daily complaints.

Q4 — Sets: Elective Registration Clash Detection (7 Marks | K4)

Problem Context: Identifying elective clashes, single enrollments, and non-registered students.

1. Python Implementation (Operators & Methods)

```
ai_elective = {"192311001", "192311004", "192311007", "192311009"}
iot_elective = {"192311002", "192311004", "192311009", "192311010"}
all_students = {f"192311{i:04d}" for i in range(1, 11)}

# TODO 1: Using set operators
both_op = ai_elective & iot_elective
exactly_one_op = ai_elective ^ iot_elective
neither_op = all_students - (ai_elective | iot_elective)

# Using set methods
both_m = ai_elective.intersection(iot_elective)
exactly_one_m = ai_elective.symmetric_difference(iot_elective)
neither_m = all_students.difference(ai_elective.union(iot_elective))

print("clash    :", sorted(both_op))
print("exactly 1:", sorted(exactly_one_op))
print("neither  :", sorted(neither_op))
```

2. Set Theory & Venn Diagrams

```
ALL STUDENTS (10) +-----------------------------------------------------+ | AI ELECTIVE IOT ELECTIVE
```

- **Automatic Duplicate Elimination:** Sets enforce uniqueness via hash table storage; inserting an existing key overwrites/ignores duplicates.
- **Membership Performance:** `x in set` operates in $O(1)$ average time, compared to $O(N)$ linear scanning in lists.

Q5 — List Comprehension: Cleaning Sensor Readings (6 Marks | K3)

Problem Context: Vaccine cold-storage temperature monitoring (Valid: $-50^\circ\text{C} \le t \le 50^\circ\text{C}$, Alarms: outside $2\text{--}8^\circ\text{C}$).

1. Implementation (3 Paradigms)

```
readings = [3.5, 4.0, 52.0, -60.0, 1.5, 9.2, 5.0]

# 1. Loop with append
clean_loop = []
for r in readings:
    if -50 <= r <= 50:
        clean_loop.append(r)

# 2. List Comprehension
clean_comp = [r for r in readings if -50 <= r <= 50]

# 3. Functional map/filter
clean_func = list(filter(lambda r: -50 <= r <= 50, readings))

# Alarm minutes using enumerate
alarm_minutes = [min_idx for min_idx, r in enumerate(readings)
                 if -50 <= r <= 50 and not (2 <= r <= 8)]
```

2. List Comprehension vs Generator Expression Analysis

Construct	Time Complexity	Memory Complexity	Use Case
List Comprehension	O(n)	O(n) [Allocates full list in RAM]	When full list access, indexing, or multiple iterations are required.
Generator Expression	O(n)	O(1) [Yields values lazily]	Streaming pipeline, single-pass aggregations (e.g., `sum()`).

Q6 — Nested Data: A Railway Timetable (7 Marks | K4 - Analyse)

Problem Context: Railway inquiry counter mapping train numbers to nested dictionary details.

1. Implementation & Index Design

```
timetable = {
    "12639": {
        "name": "Brindavan Express",
        "days": {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"},
        "stops": [("MAS", "07:40", "07:40"), ("KPD", "09:28", "09:30"), ("SBC", "13:40",
"13:40")]
    },
    "12601": {
        "name": "Mangalore Mail",
        "days": {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"},
        "stops": [("MAS", "20:15", "20:15"), ("KPD", "22:08", "22:10"), ("MAQ", "12:05",
"12:05")]
    }
}

# Expression to reach 3rd station departure time for train "12639"
dep_time_st3 = timetable["12639"]["stops"][2][2] # "13:40"

# Building Station Index: Station -> List of (train_no, arr, dep)
def build_station_index(tt):
    index = {}
    for train_no, data in tt.items():
        for stn, arr, dep in data["stops"]:
            index.setdefault(stn, []).append((train_no, arr, dep))
    return index
```

2. Structural & Trade-off Analysis

- **Data Structure Choice:** Outer `dict` gives $O(1)$ lookup by train number; inner `list` preserves order of stops; station `tuple` guarantees immutable trip coordinates; running `set` enables $O(1)$ day membership check.
- **Trade-off:** Building an inverted index takes $O(T \cdot S)$ time initially and consumes additional space, but reduces query time from $O(T \cdot S)$ scanning to $O(1)$ direct hash retrieval.

Q7 — Design a Contact-Merging Program (10 Marks | K5 - Evaluate)

Problem Context: Merging duplicate contacts from phone storage and SIM cards from first principles.

1. System Architecture & Pseudocode

```
FUNCTION NormaliseNumber(phone_str):
    REMOVE spaces, hyphens, and "+91" prefix
    RETURN last 10 digits

FUNCTION MergeContacts(contact_list):
    merged_db = {} # Key: normalised_phone -> Record Dict
    FOR contact IN contact_list:
        norm_phone = NormaliseNumber(contact.phone)
        IF norm_phone NOT IN merged_db:
            merged_db[norm_phone] = {
                "names": {contact.name},
                "phones": {contact.phone},
                "emails": {contact.email} if contact.email else set()
            }
        ELSE:
            merged_db[norm_phone]["names"].add(contact.name)
            merged_db[norm_phone]["phones"].add(contact.phone)
            IF contact.email:
                merged_db[norm_phone]["emails"].add(contact.email)
    RETURN merged_db
```

2. Defense & Complexity Comparison

Strategy	Time Complexity	Space Complexity	Evaluation
Pairwise Comparison	$O(N^2)$	O(1) aux	Requires ~10^{10} comparisons for 100,000 contacts. Unusable in production.
Hash-Key Grouping	O(N) avg	O(N)	Processes 100,000 contacts in sub-second time using hash maps. Highly scalable.

Limitation & Exception Handling: If two different people share the exact same normalized phone number (e.g., family shared phone), they will be merged into one record. Conversely, if a person has two contacts with different phone numbers and distinct names, they will remain separate records.